

9 電腦視覺的深度學習簡介

Google colab檔案: [深度學習9-2.ipynb](#) [深度學習9-3.ipynb](#) [深度學習9-4.ipynb](#)

9-1: 電腦視覺的三種基本任務

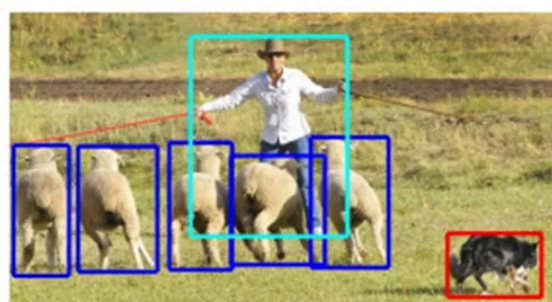
影像分類Image Classification: 目標是為一張影像指定一或多個標籤, 可能為單一標籤(一張圖只屬於一個標籤), 或多個標籤(一張圖屬於多個類別)的分類任務。

影像分割Image Segmentation: 目標是把一張圖分割或劃分成不同區塊, 每個區塊通常代表一個類別。

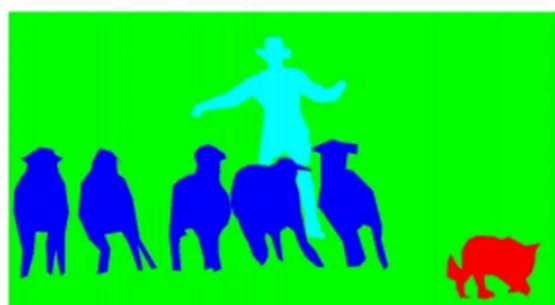
物體偵測Object Detection: 目標是在影像中加我們要的物件周圍框出一些矩型(稱為邊界框), 並為每個矩形標註一個類別。



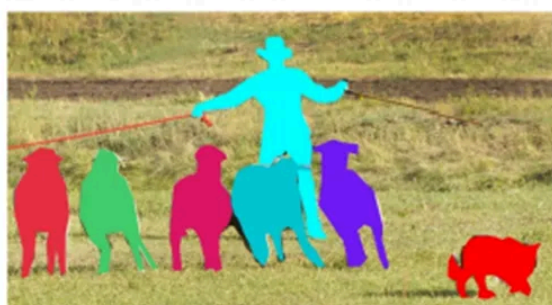
(a) Image classification



(b) Object localization



(c) Semantic segmentation



(d) Instance Segmentation

除此之外還有影像相似性評分(評估兩張影像在視覺上的相似程度), 關鍵點偵測(例如臉部特徵), 姿勢預測等等

9-2: 影像分割

指的是模型來為途中的每個像素指定一個類別

影像分割可以分為兩大類:

1. 語意分割Semantic Segmentation: 是指將圖像中的所有像素獨立歸入一個語意類別中(如貓)。
2. 實例分割Instance Segmentation: 是物件偵測和語義分割的結合, 任務相對較難。其方法是針對感興趣的像素點進行分類, 並且將各個物件定位, 即使是相同類別也會分割成不同物件(如貓1, 貓2)。



```

import matplotlib.pyplot as plt
from tensorflow.keras.utils import load_img, img_to_array

def display_target(target_array):
    # 將目標數組歸一化到0-255範圍，便於顯示。原始標註值為1, 2, 3，轉換後為0, 127, 254
    normalized_array = (target_array.astype("uint8") - 1) * 127
    plt.axis("off") # 關閉座標軸顯示
    plt.imshow(normalized_array[:, :, 0]) # 顯示處理後的標註圖像 (灰度圖只需一個通道)

# 載入第10張標註圖像 (索引為9)，並轉換為灰度圖數組
img = img_to_array(load_img(target_paths[9], color_mode="grayscale"))
display_target(img) # 顯示標註圖像

```



1. 為什麼不用「最大池化層 (MaxPooling2D)」？

在一般的影像分類(如辨識貓或狗)中，我們常用 MaxPooling。但這段程式碼選擇用跨步卷積 **Strided Convolutions**(步長大於 1 的卷積)，原因如下：

- 資訊流失：MaxPooling 只保留窗格中的最大值，會丟失掉該數值原本在 2x2 區域中的「精確座標」。
- 空間位置的重要性：影像分割需要精確知道物體的邊界在哪裡。跨步卷積(strides=2)在縮小圖片尺寸的同時，在更能保留位置資訊的情況下，對特徵圖進行降採樣。

程式碼的前半段是編碼器，其目的是「理解語義」。

2. 編碼器 (Encoder): 降採樣 (Downsampling)

- 做法: 透過連續的卷積, 將原始圖片從 $(200, 200, 3)$ 壓縮到 $(25, 25, 256)$ 。
- 結果: 雖然圖片變小了, 但每個「像素點」現在代表了原始影像中很大一塊區域的特徵。這就像是你把整本書濃縮成大綱, 雖然字變少了, 但每個字都代表了深刻的意義。

3. 解碼器 (Decoder): 升採樣 (Upsampling)

這就是你問的 **Conv2DTranspose** (轉置卷積) 的作用。

- 逆向操作: 如果說 **Conv2D** 是把圖片「變小且加深」, 那麼 **Conv2DTranspose** 就是把圖片「變大且變淺」。
- 學習如何放大: 它不是單純的插值放大 (像我們手動拉大照片會變模糊), 它是有權重 (Weights) 的。模型會學習如何根據編碼器提取的特徵, 精確地填補像素, 最終把 $(25, 25)$ 的特徵圖還原回 $(200, 200)$ 。

```
# 編碼器部分 (降採樣路徑), 我們在每個Conv2D層都用了[padding="same"], 藉此避免邊界效應對特徵圖的尺寸造成影響
x = layers.Conv2D(64, 3, strides=2, activation="relu", padding="same")(x) # 卷積層, 步長為2來縮小採樣數
x = layers.Conv2D(64, 3, activation="relu", padding="same")(x) # 普通卷積層
x = layers.Conv2D(128, 3, strides=2, activation="relu", padding="same")(x)
x = layers.Conv2D(128, 3, activation="relu", padding="same")(x)
x = layers.Conv2D(256, 3, strides=2, padding="same", activation="relu")(x)
x = layers.Conv2D(256, 3, activation="relu", padding="same")(x)

# 解碼器部分 (升採樣路徑)
x = layers.Conv2DTranspose(256, 3, activation="relu", padding="same")(x) # 反卷積層, 用於上採樣
x = layers.Conv2DTranspose(256, 3, activation="relu", padding="same", strides=2)(x) # 步長為2來縮小採樣數
x = layers.Conv2DTranspose(128, 3, activation="relu", padding="same")(x)
x = layers.Conv2DTranspose(128, 3, activation="relu", padding="same", strides=2)(x)
x = layers.Conv2DTranspose(64, 3, activation="relu", padding="same")(x)
x = layers.Conv2DTranspose(64, 3, activation="relu", padding="same", strides=2)(x)
```

9-3: 現代卷積神經網路的架構模式

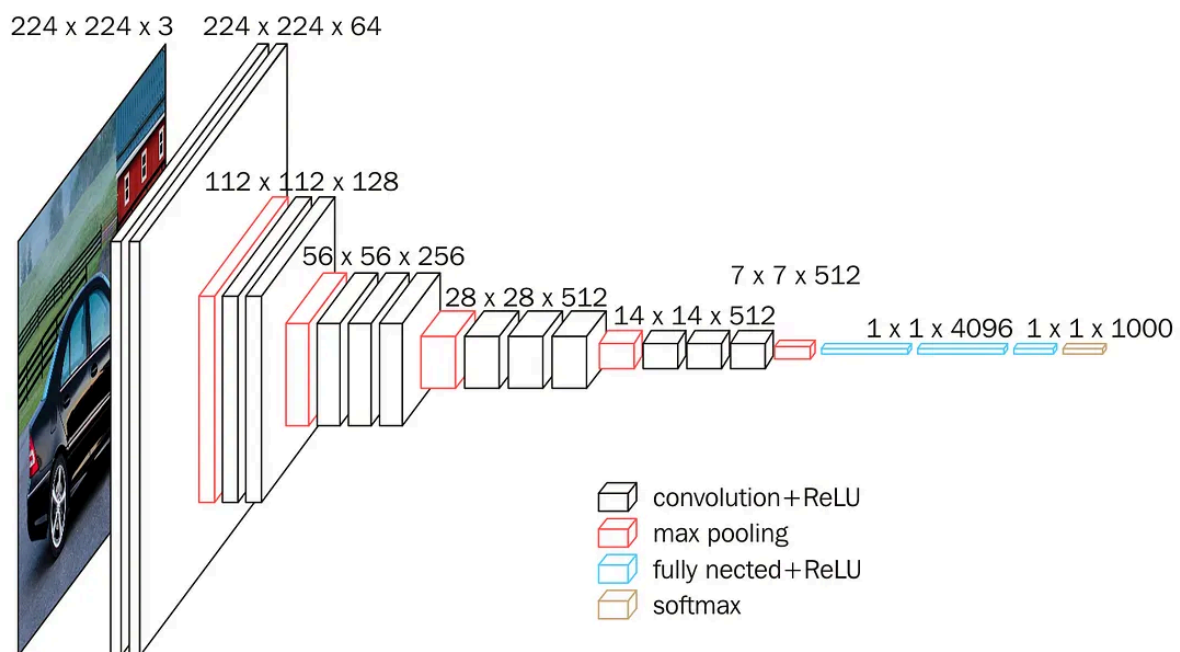
前言:

模型架構定義了假設空間, 將先驗知識編碼進模型中, 決定梯度下降的搜尋難度與模型能否有效學習。好的架構可加速收斂、降低資料需求, 而架構設計更像一門依賴經驗與直覺的藝術。

9-3-1 模組化、階層化、及重複使用(MHR)

如果想讓一個複雜的系統變簡單, 通用配方就是將複雜內容構成模組, 然後組織模組的階層結構, 並在適當的地方重複使用相同的模組。

深度學習透過梯度下降來進行連續優化。在連續的函數空間中進行梯度下降, 並將搜尋空間結構化成多個模組 (層), 最後組織成一個深度階層結構, 舉例來說 VGG16 就是由重複的「卷積、卷積、最大池化」區塊構成。



越深的階層結構在本質上會越好，因為它們能促進特徵的重複使用(萃取)，也因此能提升抽象化能力。

9-3-2 殘差連棧(residual connection)

由於梯度消失(vanishing gradient: 隨著層數的增加，往往會遇到梯度不穩定的問題。例如傳聲遊戲第一個人用耳語傳遞訊息一路到最後一個人，最終的訊息與最初版本不同)，我們堆疊層的深度是受限制的。為了解決這個問題，就要用殘差連棧(residual connection)。

假設，如果你多堆一層，但這一層的結果和上一層一樣，從邏輯上來說，這樣基本上當然不會出現退化是也。這一層，我們叫做 Identity mapping, 恆等映射。

讓我們先建立以下的概念：

輸入是 x ，學到的特徵是 $H(x)$ ，殘差 $\text{Residual} = H(x) - x$

殘差的概念很直覺吧？就是學到的特徵和原本的輸入的差異。

既然是 $\text{Residual} = H(x) - x$ ， Residual 也是 x 的函數，所以也寫作 $F(x)$ ，那 $F(x) = H(x) - x$

原本學習是這樣的。 $x \rightarrow H(x)$

已經知道 $F(x) = H(x) - x$

所以學習也可以這樣寫： $x \rightarrow F(x) + x$

因為 $H(x) = F(x) + x$

用文字來說明的話，

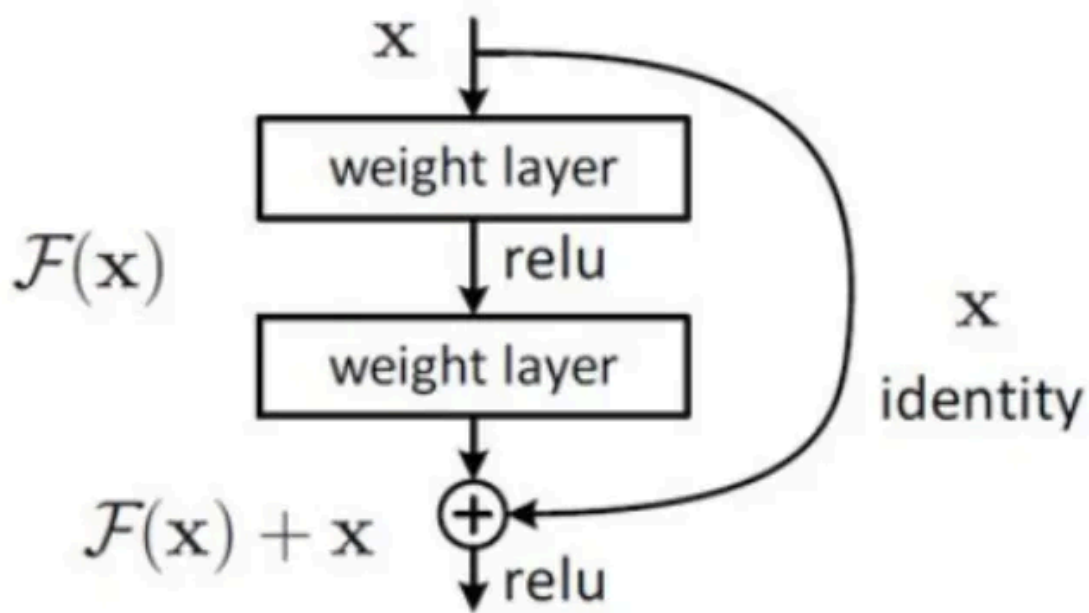
輸入→特徵

變成：輸入→輸入 + 殘差

這樣有什麼好處嗎？

如果今天我們多一層，什麼都沒有學到。那殘差就是0

那多這一層，想想我們上面介紹過的，當殘差=0的時候，輸入→輸入，這一層就叫做恆等映射，因此，多這一層如果沒學到新的特徵，也不會讓模型退化。實際上當然不會剛好等於0，而是可以增加很多層，而每一層都可以學到一些新的更複雜的特徵。



```
inputs = keras.Input(shape=(32, 32, 3))
x = layers.Rescaling(1./255)(inputs)

def residual_block(x, filters, pooling=False):
    residual = x # 1. 儲存原始輸入, 作為殘差路徑的起點
    x = layers.Conv2D(filters, 3, activation="relu", padding="same")(x)
    x = layers.Conv2D(filters, 3, activation="relu", padding="same")(x)
    if pooling:
        x = layers.MaxPooling2D(2, padding="same")(x)
        residual = layers.Conv2D(filters, 1, strides=2)(residual) # 2. 調整殘差路徑 (池化時)
    elif filters != residual.shape[-1]:
        residual = layers.Conv2D(filters, 1)(residual) # 2. 調整殘差路徑 (過濾器數量改變時)
    x = layers.add([x, residual]) # 3. 將主路徑輸出與殘差相加
    return x

x = residual_block(x, filters=32, pooling=True)
x = residual_block(x, filters=64, pooling=True)
x = residual_block(x, filters=128, pooling=False)

x = layers.GlobalAveragePooling2D()(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs=inputs, outputs=outputs)
model.summary()
```

9-3-3 批次正規化

正規化的目的 1. 讓不同特徵尺度更接近 2. 使資料分佈穩定 3. 幫助模型更容易學習並提升泛化能力	常見作法有 1. 使資料平均值 = 0 2. 標準差 = 1 (變異數 = 1) 3. 假設資料近似常態分佈
--	---

即使「輸入資料」已正規化，經過 Dense / Conv 層後，輸出分佈仍會改變，因此需要對「中間層輸出」進行正規化。

批次正規化：由 Ioffe & Szegedy (2015) 提出，功能：在訓練過程中，對每一層的輸出進行動態正規化。

運作方式：

- 在訓練期間，批次正規化會用當前批次資料的平均值和變異數來進行正規化。另外還會以指數移動平均 (exponential moving average) 的算法，用每一批次的平均值和變異數來持續更新 2 個層參數：移動平均值及移動變異數。
- 在推論期間，由於無法取得足夠大的批次資料來計算具「整體代表性」的平均值與變異數，所以它會改用訓練時所算出、具整體代表性的移動平均值及移動變異數來做正規化。

雖然其理論機制尚未完全釐清，但已成為現代深度學習的標準技巧。

實務上的主要好處：

1. 改善梯度傳播
2. 穩定訓練過程
3. 加速收斂
4. 讓「更深層網路」可以成功訓練

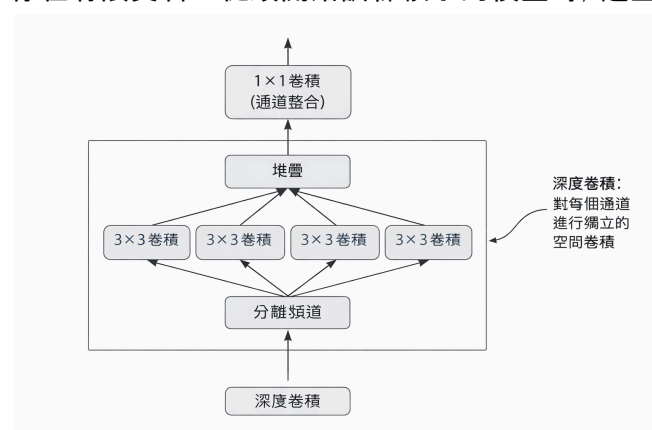
```
x = layers.Conv2D(32, 3, use_bias=False)(x) #此處先不使用激活函數
x = layers.BatchNormalization()(x)
x = layers.Activation("relu")(x) #經過正規化後再使用激活函數
```

9-3-4 深度可分離卷積

其目的為讓模型更小 (更少的可訓練權重參數、更精簡 (更少的浮點數運算)、還能讓模型的表現提升幾個百分點。

透過逐點卷積 (pointwise convolution, 一個 1×1 的卷積) 分開學習「空間特徵」和「逐個 channel 的特徵」。

與卷積操作假設「影像中的 pattern 不會受限於特定位置」一樣，深度可分離卷積假設「中間激活結果 (intermediate activation) 中的空間位置非常相關，但不同 channel 之間則非常獨立」。跟一般的卷積相比，深度可分離卷積需要的參數和計算量都少了許多，但表徵能力 (representational power) 卻很強。它讓模型更小、收斂速度更快，也更不容易出現過度配適。當你在有限資料上從頭開始訓練較小的模型時，這些優勢就顯得特別重要。



9-4: 卷積神經網路學到了甚麼？

9-4-1 視覺化中間層的激活效果

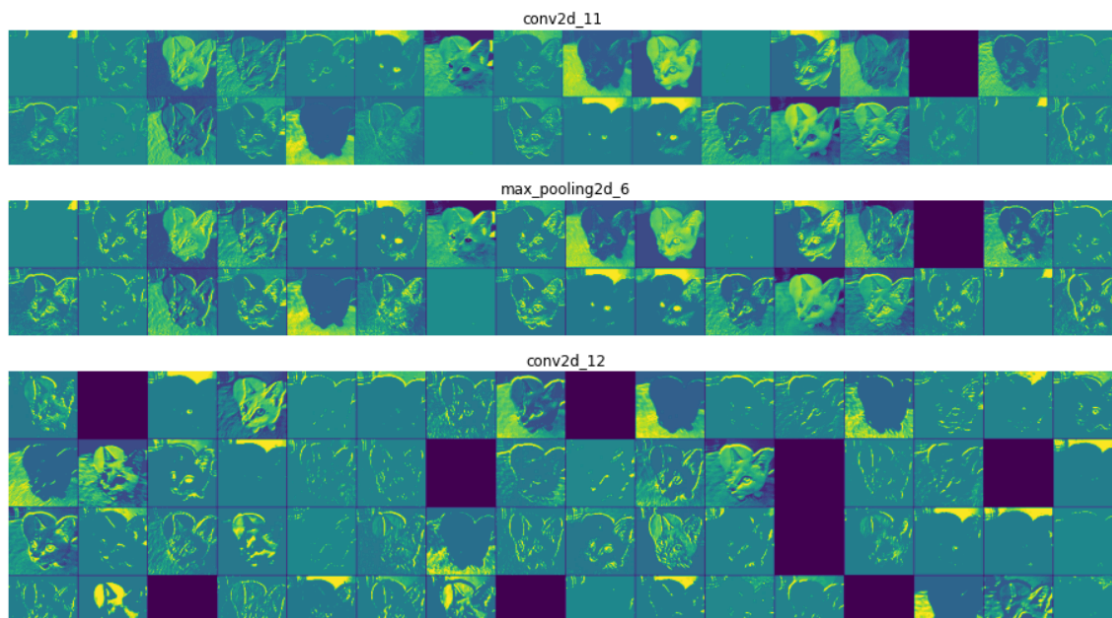
有助於我們理解連續的卷基層如何轉換其輸入，也能幫我們初步理解個卷積網路過濾器的意義。

```
for layer_name, layer_activation in zip(layer_names, activations):
    n_features = layer_activation.shape[-1] # 取得特徵的數量 (即 channel 數量)
    size = layer_activation.shape[1] # 取得激活圖的高度/寬度
    n_cols = n_features // images_per_row # 計算需要顯示的列數
    # 建立一個空的網格來存放所有 channel 的圖像
    display_grid = np.zeros((size + 1) * n_cols - 1,
                             images_per_row * (size + 1) - 1))

    # 遍歷每個 channel 並將其添加到網格中
    for col in range(n_cols):
        for row in range(images_per_row):
            channel_index = col * images_per_row + row # 計算當前 channel 的索引
            channel_image = layer_activation[0, :, :, channel_index].copy() # 取得當前 channel 的激活圖
            # 對激活圖進行後處理以使其可視化
            if channel_image.sum() != 0:
                channel_image -= channel_image.mean() # 減去均值
                channel_image /= channel_image.std() # 除以標準差
                channel_image *= 64 # 縮放
                channel_image += 128 # 加上偏移量
            channel_image = np.clip(channel_image, 0, 255).astype("uint8") # 將像素值裁剪到 [0, 255] 並轉換為 uint8 類型
            # 將處理後的 channel 圖像放到 display_grid 中
            display_grid[
                col * (size + 1): (col + 1) * size + col,
                row * (size + 1) : (row + 1) * size + row] = channel_image

    scale = 1. / size # 計算縮放比例
    # 建立一個新的 figure 來顯示網格
    plt.figure(figsize=(scale * display_grid.shape[1],
                        scale * display_grid.shape[0]))

    plt.title(layer_name) # 設定標題為當前層的名稱
    plt.grid(False) # 隱藏網格
    plt.axis("off") # 隱藏坐標軸
    plt.imshow(display_grid, aspect="auto", cmap="viridis") # 顯示圖像網格
```



其結果顯示每層激活結果中的每個channel

```
plt.imshow(display_grid, aspect="auto", cmap="viridis")
```

直接呈現「連續卷積層如何逐步轉換輸入」

- 前面幾層:通常看到邊緣、角落、簡單紋理。

- 中間層:開始出現局部形狀、輪廓。
- 後面層:變得越來越抽象, 只對高階語意區域有反應。

9-4-2視覺化卷積神經網路的過濾器

呈現每個過濾器會對什麼視覺 pattern 做出反應。我們可以透過在輸入空間應用梯度上升 (gradient ascent) 來達成這件事:

從一張空白的輸入影像開始, 不斷以梯度下降法來調整輸入影像的值, 以讓特定過濾器的響應值 (response) 達到最大(編註:只會改變輸入影像, 而不會改變過濾器中已訓練好的參數)。
*梯度下降法一般是用來讓輸出的loss 最小, 但這裡是要讓輸出的值為最大, 因此將之稱為梯度上升。

重要程式碼

程式 9.15:loss 定義

```
import tensorflow as tf

def compute_loss(image, filter_index):
    # 使用特徵提取器模型獲取輸入圖像的激活 (特徵圖)
    activation = feature_extractor(image)
    # 從激活中選擇特定濾波器 (由 filter_index 指定) 的特徵圖,
    # 並裁剪掉邊緣的2個像素 (通常是為了避免邊界效應或聚焦於核心特徵)
    filter_activation = activation[:, 2:-2, 2:-2, filter_index]
    # 計算所選濾波器特徵圖的平均激活值, 這將作為損失 (我們希望最大化這個值)
    return tf.reduce_mean(filter_activation)
```

這個函式就是我們在梯度上升過程中要最大化的函式, 這邊使用 `model(x)` 而不是 `predict()`, 其原因為 `predict()` 可以處理非常大的陣列, 另一方面 `model(x)` 則是將整個 `x` 都載到記憶體內運算, 因此無法處理太大的陣列。不過 `predict` 是不可微分的如果在 `GradientTape` 範圍內呼叫它, 將無法取得梯度資訊。

若你需要取得呼叫模型後的梯度, 就要用 `model(x)`

若只需要輸出值, 就用 `predict()`。

換句話說, 除非你正在寫一個底層的梯度下降迴圈, 否則都是使用 `predict`。

程式 9.16: 梯度上升更新影像

```
@tf.function
def gradient_ascent_step(image, filter_index, learning_rate):
    with tf.GradientTape() as tape:
        tape.watch(image) #由於image張量不是TensorFlow的Variable物件, 所以要使用watch()追蹤其內部值 (在梯度磁帶中, 只有Variable物件會被自動追蹤)
        loss = compute_loss(image, filter_index) #計算損失值 (為一純量, 代表當前影像激活過濾器的程度)
    grads = tape.gradient(loss, image) #計算損失相對於影像的梯度
    grads = tf.math.l2_normalize(grads) #使用梯度正規化技巧
    image += learning_rate * grads
    return image
```

`image += learning_rate * grads` 用隨機梯度上升法來最大化損失

程式 9.17 的核心 loop

```
for i in range(iterations):
    image = gradient_ascent_step(image, filter_index,
learning_rate) #重複更新影像張量的值, 藉此最大化損失函數值
```



```

img_width = 200
img_height = 200

def generate_filter_pattern(filter_index):
    iterations = 30 # 要進行梯度上升的步數
    learning_rate = 10. # 每一次梯度上升的幅度
    image = tf.random.uniform(
        minval=0.4,
        maxval=0.6,
        shape=(1, img_width, img_height, 3)) # 初始化一個隨機影像張量, (由於Xception模型預期輸入值在[0,1]範圍內, 所以我們在這裡選擇一個以0.5為中心的範圍)
    #重複更新影像張量的值, 藉此最大化損失函數值
    for i in range(iterations):
        image = gradient_ascent_step(image, filter_index, learning_rate)
    return image[0].numpy()

```

9-4-3 視覺化類別激活熱力圖

理解影像中的那些部分引導了卷積神經網路最終的分類決策

視覺化類別激活圖 (class activation map, CAM), 它可在輸入影像上生成類別激活結果的熱力圖。類別激活熱力圖是一個與特定輸出類別有關的 2D 分數網格, 它會針對輸入影像中的每個位置做計算, 並指出每個位置對特定類別的重要性。例如, 把一張圖餵入卷積神經網路, CAM 視覺化就能針對「貓」這個類別產生一個熱力圖, 指出影像各部分很像是「貓」的程度。Grad-CAM 的運作方式為: 給定一張輸入影像後, 取得最後一層卷積層輸出的特徵圖, 用特定類別相對於特徵圖中每個 channel 的梯度, 對所有 channel 這行加權。直觀而言, 就是想像自己正在用「每個特徵圖 channel 對該類別的重要程度」, 對「輸入影像對每個特徵圖」輸入影像對該類別的激活程度」。

整理運作:

1. 先對已訓練好的模型輸入一張影像(A)做推論:
輸入影像(A) → [各卷積層] → 含多個 channel 的特徵圖(B) → [分類器] → 機率最高的類別(C)
2. 接著反向算出特徵圖(B)中的每個 channel, 對類別(C)的重要性(W)。
3. 然後反向算出影像(A)中的每個位置, 對特徵圖(B)中每個 channel 的重要性(V)。
4. 最後將重要性(W) 和(V)相乘, 即可算出影像(A)中的每個位置, 對類別(C)的重要性, 並以此在影像(A)中繪製熱力圖。

重要程式碼:

程式 9.24: 取得目標類別對最後卷積層的梯度

```

import tensorflow as tf

with tf.GradientTape() as tape:
    #計算最後一個卷積層的激活結果, 並指定要追蹤
    last_conv_layer_output = last_conv_layer_model(img_array)
    tape.watch(last_conv_layer_output)
    #取得最高分類別的激活channel
    preds = classifier_model(last_conv_layer_output)
    top_pred_index = tf.argmax(preds[0])
    top_class_channel = preds[:, top_pred_index]
    #計算[最高分的預測類別]相對於「最終卷積層之輸出特徵圖」的梯度
    grads = tape.gradient(top_class_channel, last_conv_layer_output)

```

grads = tape.gradient(top_class_channel, last_conv_layer_output)

程式 9.25 梯度池化及channel重要性加權

```

pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2)).numpy() #得到每個channel對最高分類別的重要權重
last_conv_layer_output = last_conv_layer_output.numpy()[0]
for i in range(pooled_grads.shape[-1]):
    #把最後一個卷積層輸出中的每個channel乘以[該channel的重要程度]，藉此加權
    last_conv_layer_output[:, :, i] *= pooled_grads[i]
heatmap = np.mean(last_conv_layer_output, axis=-1)
#把所有加權後的channel平均，得到類別激活熱力圖

```

pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2)).numpy():梯度池化

last_conv_layer_output[:, :, i] *= pooled_grads[i]:加權最後卷積層輸出

heatmap = np.mean(last_conv_layer_output, axis=-1):把所有加權後的channel平均，得到類別激活熱力圖